Mikrodatorteknik (TMIK13) – Tenta C med svar

Fil: GetInsperaExamPdf 2.pdf · 14 uppgifter · 50 p

> **Hur du läser:** Varje uppgift har först **Svar**, sedan en kort *Förklaring* på enkel svenska. Skriv gärna egna kommentarer i filen.

1. Funktion add_mul (2 p)

```
float add_mul(float a) { return 2/a + 1; }
float b = add_mul(0.2);
```

Svar: b = 11.0 · Flytta till annan .c-fil: **Falskt** (b får inte samma värde).

Förklaring:

- 2 / 0.2 = 10, plus 1 = **11**.
- Flyttar man funktionen till en annan .c-fil **utan prototyp** (`float add_mul(float a);` i en .h-fil som inkluderas) vet main.c inte hur funktionen ser ut. C antar då att den returnerar **int** och skickar argumentet som **double** → funktionen läser fel värde och main läser returvärdet från fel ställe → **b blir fel**.
- *Lärdom:* lägg alltid prototypen i en header-fil.

2. Assembler (3 p)

```
my_function:
  MOVS R0, #255
0x8001644:
  NOP
  NOP
  SUBS R0, R0, #1
  BNE 0x8001644
```

Fråga	Svar
R0 initialt	**255**
SUBS R0, R0, #1	**Subtraherar 1 från R0 och uppdaterar flaggor**
BNE beror på	**Zero**
Antal NOP	**510**
När R0 = 0	**Programmet fortsätter**
Syfte	**Åstadkomma en kort fördröjning**

Förklaring:

- Loopen körs 255 gånger (255 → 1). Varje varv har **2 NOP** → 255 × 2 = **510**.
- **S** i SUBS = uppdatera flaggor. **BNE** = hoppa om Z = 0 (resultatet ≠ 0).
- När R0 blir 0 sätts Z = 1 → ingen hoppning → programmet går vidare.

3. ADC – strömsensor (1 p)

VOUT = 0 V vid 0 A, +0.25 V per ampere. Digitalt värde 2548.

Svar: ≈ 8.21 A

Förklaring:

1. U = 2548 / 4095 × 3.3 = **2.053 V**
2. I = 2.053 / 0.25 = **8.21 A**

Grundformel ADC: **U = värde / 4095 × 3.3 V** (vissa lärare använder 4096 – ger nästan samma svar).

4. PLL (2 p)

Svar: PLL = **Phase-Locked Loop** (faslåst slinga). Den används för att **multiplicera upp** en låg klockfrekvens till en högre systemklocka.

Hur den fungerar: En VCO (spänningsstyrd oscillator) genererar en hög frekvens. Den delas ner och jämförs i en **fasdetektor** med referensklockan. Skillnaden filtreras och styr VCO:n så att den låser sig på exakt rätt frekvens.

I STM32L433:

- Klockkälla: HSI16 (16 MHz), MSI eller HSE (extern kristall).
- Först delas den med **M**, sedan multipliceras med **N** (i VCO:n), sedan delas med **R** (systemklocka), **P** och **Q** (andra utgångar, t.ex. USB/SAI).
- f_ut = f_in / M × N / R. Exempel: 16 MHz / 1 × 10 / 2 = **80 MHz** (max för L433).

Förklaring: Kristaller och interna oscillatorer finns bara för låga frekvenser, men CPU:n vill gå snabbt. PLL:en “multiplicerar” klockan och ser till att den ändå är lika stabil som referensen.

5. GPIO-register (3 p)

Svar:

- **MODER** (mode): väljer läge för varje pinne – **Input, Output, Alternate function** (t.ex. UART, TIM, I2C) eller **Analog** (ADC/DAC). 2 bitar per pinne.
- **OTYPER** (output type): **push-pull** eller **open-drain** för utgången.
- **OSPEEDR** (output speed): hur snabbt utgången kan slå om (low/medium/high/very high). Högre = snabbare flanker men mer störningar/ström.
- **PUPDR** (pull-up/pull-down): slår på intern **pull-up**, **pull-down** eller ingen.
- **IDR** (input data register): **läsa** pinnens nivå (0/1).
- **ODR** (output data register): **skriva** vad utgången ska ge (0/1).

Minnesregel: MODE = vad är pinnen, TYPE = hur driver den, SPEED = hur snabbt, PUPD = motstånd, I = in, O = ut.

6. NVIC (1 p)

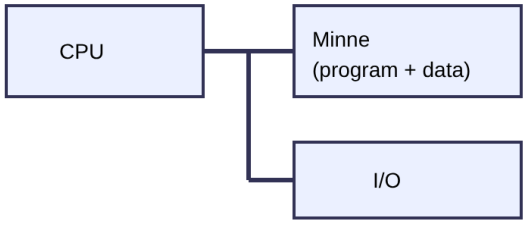
Svar: NVIC = **Nested Vectored Interrupt Controller**.

Förklaring: Det är den del i ARM Cortex-M som hanterar alla avbrott (interrupts). “Nested” = ett avbrott med högre prioritet kan avbryta ett avbrott med lägre prioritet. “Vectored” = varje avbrott har en egen adress i en tabell (vektortabellen), så CPU:n hoppar direkt till rätt ISR.

7. Harvard och von Neumann (2 p)

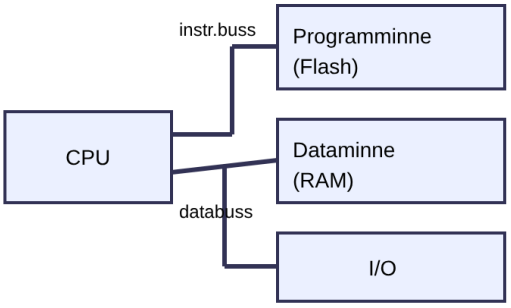
Svar:

von Neumann (en buss för allt)



En gemensam adress-/databuss.
CPU kan bara hämta instruktion
ELLER data åt gången (flaskhals).

Harvard (separata bussar)



Instruktion och data kan hämtas
samtidigt (snabbare). STM32/ARM Cortex-M = Harvard.

- **von Neumann:** program och data ligger i **samma minne** och delar **en buss**. Enkelt men CPU:n kan inte hämta instruktion och data samtidigt (“von Neumann-flaskhalsen”).
- **Harvard: separata minnen och bussar** för program (instruktioner) och data. CPU:n kan hämta nästa instruktion samtidigt som den läser/skriver data → snabbare. ARM Cortex-M (STM32) bygger på Harvard-principen.

Tips på tentan: Rita rutor för CPU, minne(n) och I/O, och visa tydligt **en** buss vs **två** bussar.

8. DMA (2 p)

Svar: DMA = **Direct Memory Access**.

En DMA-styrenhet kan flytta data **utan att CPU:n gör jobbet**: mellan kringenhet ↔ minne (t.ex. ADC → RAM, RAM → UART/SPI) eller minne ↔ minne. CPU:n ställer bara in källa, destination, antal och sedan sköter DMA:n överföringen. När den är klar kan den ge ett avbrott (t.ex. “transfer complete”, även “half transfer”).

Användningsområden:

- ADC som samplar många värden kontinuerligt → DMA lägger dem i en buffert i RAM.
- Skicka/ta emot mycket data på UART, SPI, I2C (t.ex. en display, SD-kort).
- Ljud (DAC/I2S) där data måste komma i jämn takt.
- Kopiera stora minnesblock.

Fördel: CPU:n blir avlastad – kan göra annat eller sova (spar ström), och överföringen går snabbt och i jämn takt.

Förklaring: Tänk dig DMA som en “flyttgubbe”: CPU:n säger “flytta 100 värden från ADC till den här arrayen” och kan sedan göra annat.

9. Pull-up / pull-down (2 p)

Svar:

- **Pull-up:** ett motstånd mellan pinnen och **VCC (3.3 V)**. När inget annat driver pinnen läses den som **1 (hög)**.
- **Pull-down:** ett motstånd mellan pinnen och **GND**. När inget annat driver pinnen läses den som **0 (låg)**.

När används det? När en ingång annars skulle “flyta” (vara odefinierad), t.ex. en knapp som bara kopplar till GND (då behövs pull-up), eller en sensor med **open-drain/open-collector**-utgång som bara kan dra ner till GND. Även I2C-linjer behöver pull-up.

Hur aktiveras det? Antingen med ett yttre motstånd (typ 4.7–10 kΩ) eller med det **inbyggda** motståndet i STM32: registret **PUPDR** (i CubeMX: GPIO → “Pull-up” / “Pull-down”).

Förklaring: En ingång som inte är kopplad till något är som en antenn – den kan läsas som 0 eller 1 slumpmässigt. Motståndet ger ett “standardläge” men är så stort att knappen/sensorn lätt kan ändra nivån.

10. RAM-minne (3 p)

Figuren: 44-pin TSOP II med A0–A17, I/O0–I/O15.

Fråga	Svar
Adresspinnar	**18** (A0–A17)
Ordlängd	**16 bitar** (I/O0–I/O15)
Antal byte	**524288**

Förklaring: 2^18 = 262 144 adresser. Varje adress har 16 bitar = 2 byte → 262 144 × 2 = **524 288 byte**. /BHE, /BLE, /CE, /OE, /WE är styrsignaler, inte adresser.

11. PWM-styrning (5 p)

Givet: 80 MHz, PWM ≈ 5 kHz, 100 steg.

Register	Svar
PSC	**159**
ARR	**99**
CCR1 (25 %)	**25**
PSC för max frekvens	**0**
Frekvens	**800** kHz

Förklaring:

- 100 steg → räknaren räknar 0…99 → **ARR = 100 − 1 = 99**.
- Räknaren måste då gå med 5 kHz × 100 = 500 kHz. 80 MHz / 500 kHz = 160 → **PSC = 160 − 1 = 159**.
- Duty = CCR / (ARR+1) → 25 % av 100 = **25**.
- Max frekvens: dela inte alls → PSC = 0 → 80 MHz / 100 = **800 kHz** (långt över hörbart).

Formel att kunna: **f_PWM = f_clk / ((PSC+1)·(ARR+1))**. Kom ihåg "−1" på både PSC och ARR!

12. Sant/falskt om timers (3 p)

Svar (bocka i dessa – de är SANNA):

- ✔ Vissa timers kan räkna även när CPU sover, om deras klockkälla är aktiv. *(t.ex. LPTIM, RTC)*
- ✔ Auto-reload registret (ARR) bestämmer vid vilket värde timern återgår till 0.
- ✔ När räknaren slår om kan en interrupt genereras om den är aktiverad. *(update event)*

FALSKA (bocka inte):

- ✘ “En timer har alltid samma frekvens som CPU-kärnan” – nej, den kan ha annan klocka och delas med prescalern.
- ✘ “Prescalern används för att öka counterns räkningshastighet” – nej, prescalern **delar** klockan → räknaren går **långsammare**.
- ✘ “PWM kan generera en analog spänning direkt” – nej, PWM är bara 0/3.3 V-pulser. Man behöver ett lågpassfilter (RC) för att få en analog medelspänning.

Förklaring: Felbockning ger −1, så bocka bara det du är säker på.

13. Timer-avbrott (1 p)

PSC = 1249, 80 MHz, interrupt var 5 ms (200 ggr/s).

Svar: ARR = 319

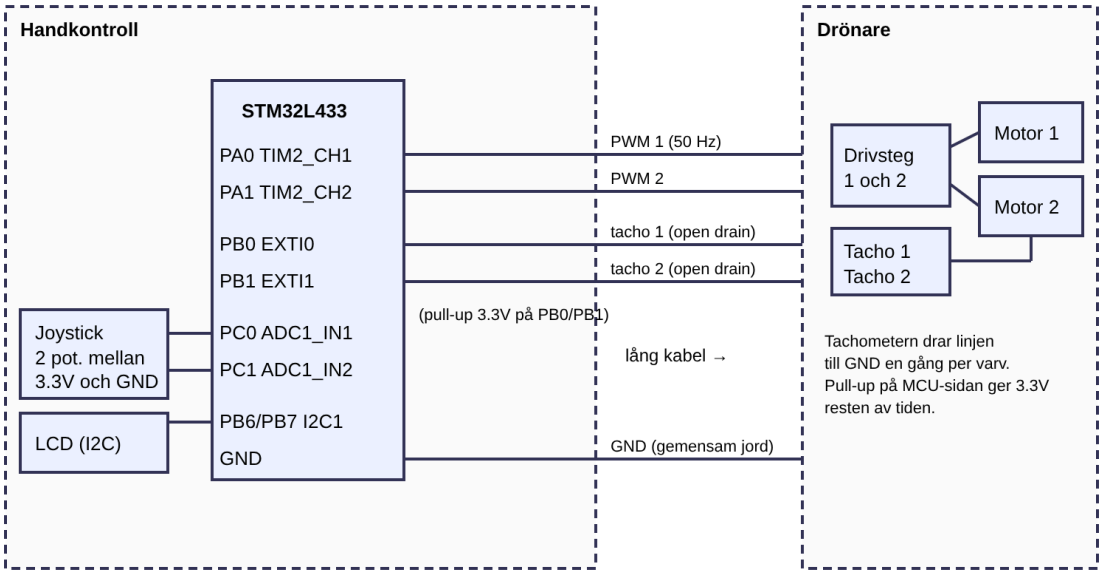
Förklaring:

- Timerklocka = 80 MHz / (1249 + 1) = **64 kHz**.
- 5 ms → 64 000 × 0.005 = 320 tick → **ARR = 320 − 1 = 319**.

14. Undervattensdrönare (20 p)

Antaganden: Joystick x på PC0 (ADC1_IN1), y på PC1 (ADC1_IN2), 12-bit ADC, mitten ≈ 2048. LCD via I2C1 (PB6/PB7) med `lcd_set_cursor()` och `lcd_print()`. Motor 1 = vänster (CH1), motor 2 = höger (CH2).

a) Kopplingsschema (3 p)



Förklaring: Viktigt att få med: PWM-ledningarna ut till drivstegen, tachometrarna in till PB0/PB1 med **pull-up** (open-drain!), joystickens potentiometrar mellan 3.3 V och GND med mittuttaget till PC0/PC1, LCD:n, och **gemensam GND** genom kabeln (annars har signalerna ingen gemensam referens).

b) PB0 och PB1 (1 p)

Svar: GPIO_EXTI0 / GPIO_EXTI1, trigger på **fallande flank**, med **pull-up** (intern via PUPDR, eller hellre ett externt motstånd t.ex. 4.7 kΩ eftersom kabeln är lång). Aktivera EXTI0 och EXTI1 i NVIC.

Förklaring: Open-drain kan bara dra ner till GND – utan pull-up skulle linjen "flyta" när den inte drar ner. Varje puls = en fallande flank = ett avbrott.

c) TIM2-konfiguration (2 p)

Inställning	Värde
Kanaler	CH1 och CH2 i **PWM Generation**, PWM mode 1
PSC	**0** (räknaren går på fulla 80 MHz)
ARR (Counter Period)	**1 599 999**
CCR (pulse) start	**120 000** (= 1.5 ms, motor står still)
Min / max CCR	**40 000** (0.5 ms) / **200 000** (2.5 ms)

Förklaring:

- 80 MHz / 50 Hz = 1 600 000 tick per period → ARR = 1 600 000 − 1. Det ryms i TIM2 (32 bitar), men **inte** i en 16-bitars timer.
- 1 tick = 1/80 MHz = 12.5 ns. 1 ms = 80 000 tick. Alltså 0.5 ms = 40 000, 1.5 ms = 120 000, 2.5 ms = 200 000.
- PSC = 0 ger **bästa upplösning** – 160 000 olika steg mellan full bakåt och full framåt.
- Starta med `HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1)` och `..._CHANNEL_2`.

d) Mäta tid mellan tachopulser (3 p)

Svar (i ord):

1. Ställ PB0/PB1 som **EXTI** med fallande flank (se b) och aktivera avbrottet i NVIC.
2. Varje gång en puls kommer anropas `HAL_GPIO_EXTI_Callback()`. Där läser man tiden med **`HAL_GetTick()`** (SysTick ger 1 ms upplösning, räcker här).
3. Tiden mellan pulserna = **nu − förra tiden**. Spara den, och spara "nu" som "förra tiden" till nästa puls. En variabel per motor (kolla vilken pinne som gav avbrottet).
4. Hastighet = 1/T → RPS = 1000 / T_ms.
5. Om ingen puls kommit på länge (t.ex. 2 s) → motorn står still, visa 0.

Alternativ: en egen timer på 1 kHz som räknar upp en ms-räknare, eller timerns **Input Capture** (mer exakt).

```
volatile uint32_t last_pulse[2] = {0, 0};
volatile uint32_t period_ms[2]  = {0, 0};

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
```

```
  int m;
  if (GPIO_Pin == GPIO_PIN_0)      m = 0;   // motor 1
  else if (GPIO_Pin == GPIO_PIN_1) m = 1;   // motor 2
  else return;

  uint32_t now = HAL_GetTick();
  period_ms[m]  = now - last_pulse[m];
  last_pulse[m] = now;
}

float get_rps(int m) {
  if (period_ms[m] == 0 || HAL_GetTick() - last_pulse[m] > 2000)
    return 0.0f;                  // står still
  return 1000.0f / period_ms[m];  // varv per sekund
}
```

f) calc_and_update_pwm(int x, int y) (7 p)

Idé: “Tank-styrning” (differentialstyrning).

- **y** (framåt/bakåt) ger **samma** fart på båda motorerna.
- **x** (sidled) **adderas** på ena motorn och **subtraheras** på den andra → drönaren svänger.
- Vänster = y + x, Höger = y − x. Begränsa (clamp) så att det inte går utanför ±max.
- Gör om till pulsbredd: 1.5 ms ± 1 ms → CCR = 120 000 ± 80 000.
- Skriv till **TIM2->CCR1** och **TIM2->CCR2**.

```
#include <stdlib.h>   // abs()

extern TIM_HandleTypeDef htim2;

#define ADC_MID      2048     // joystick i mitten
#define DEADZONE     100      // ignorera små rörelser runt mitten
#define PULSE_MID    120000   // 1.5 ms -> stilla
#define PULSE_RANGE  80000    // ±1.0 ms -> 0.5 ms .. 2.5 ms

static int32_t clamp(int32_t v, int32_t lo, int32_t hi) {
  if (v < lo) return lo;
  if (v > hi) return hi;
  return v;
}

void calc_and_update_pwm(int x, int y) {
  int32_t dx = x - ADC_MID;     // -2048 .. +2047  (sväng)
  int32_t dy = y - ADC_MID;     // -2048 .. +2047  (fram/bak)

  if (abs(dx) < DEADZONE) dx = 0;
  if (abs(dy) < DEADZONE) dy = 0;

  int32_t left  = clamp(dy + dx, -2048, 2047);
  int32_t right = clamp(dy - dx, -2048, 2047);

  uint32_t ccr1 = PULSE_MID + left  * PULSE_RANGE / 2048;  // 40000..~200000
  uint32_t ccr2 = PULSE_MID + right * PULSE_RANGE / 2048;

  __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, ccr1);   // = TIM2->CCR1
  __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_2, ccr2);   // = TIM2->CCR2
}
```

Exempel för att förstå:

- Joystick rakt fram (y max, x mitt): båda motorer ≈ 2.5 ms → full fart framåt.
- Joystick åt höger (x max, y mitt): vänster fram, höger bak → snurrar på stället.
- Joystick i mitten: båda 1.5 ms → stilla.

g) Main-loop (4 p)

```c
#include <stdio.h>
#include "lcd.h"

extern ADC_HandleTypeDef hadc1;

static uint32_t read_adc(uint32_t channel) {
  ADC_ChannelConfTypeDef s = {0};
  s.Channel      = channel;
  s.Rank         = ADC_REGULAR_RANK_1;
  s.SamplingTime = ADC_SAMPLETIME_47CYCLES_5;
  s.SingleDiff   = ADC_SINGLE_ENDED;
  s.OffsetNumber = ADC_OFFSET_NONE;
  HAL_ADC_ConfigChannel(&hadc1, &s);

  HAL_ADC_Start(&hadc1);
  HAL_ADC_PollForConversion(&hadc1, 10);
  uint32_t v = HAL_ADC_GetValue(&hadc1);
  HAL_ADC_Stop(&hadc1);
  return v;
}

int main(void) {
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  MX_ADC1_Init();
  MX_I2C1_Init();
  MX_TIM2_Init();

  __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, PULSE_MID);  // stilla
  __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_2, PULSE_MID);
  HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
  HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_2);
  lcd_init();

  uint32_t last_pwm = 0, last_lcd = 0;
  char buf[20];

  while (1) {
    uint32_t now = HAL_GetTick();

    if (now - last_pwm >= 20) {          // 50 ggr/s = samma som PWM
      last_pwm = now;
      int x = read_adc(ADC_CHANNEL_1);   // PC0
      int y = read_adc(ADC_CHANNEL_2);   // PC1
      calc_and_update_pwm(x, y);
    }

    if (now - last_lcd >= 250) {         // LCD 4 ggr/s
      last_lcd = now;
      int r1 = (int)(get_rps(0) * 10);   // en decimal utan float-printf
      int r2 = (int)(get_rps(1) * 10);
      lcd_set_cursor(0, 0);
      sprintf(buf, "M1: %d.%d RPS  ", r1 / 10, r1 % 10);
      lcd_print(buf);
      lcd_set_cursor(1, 0);
      sprintf(buf, "M2: %d.%d RPS  ", r2 / 10, r2 % 10);
      lcd_print(buf);
```

```
        }
      }
    }
```

Förklaring: Main-loopen gör två saker med olika takt: läser joysticken och uppdaterar PWM 50 ggr/s, och skriver hastigheten på LCD 4 ggr/s. Hastigheten själv mäts i bakgrunden av avbrotten (d). Ingen `HAL_Delay` → inget blockerar.